

Kubernetes Deployments

A Complete, Beginner-Friendly Guide

1. What is a Deployment?

A Deployment is a Kubernetes object that manages a set of identical Pods running your application. Instead of creating and tracking individual Pods by hand, you describe the desired state — which container image to run, how many copies (replicas) you want, and how updates should happen — and the Deployment Controller continuously works to make the actual cluster state match that desired state.

Think of a Deployment as a manager that never sleeps: if a Pod crashes, gets deleted, or the node it runs on fails, the Deployment notices the mismatch between desired and actual state and creates a replacement Pod automatically.

2. Why Deployments Exist

Running bare Pods directly has serious limitations:

- No self-healing — if a Pod dies, nothing recreates it.
- No easy way to scale — you'd have to manually create more Pods.
- No safe way to roll out a new version — updating means deleting and recreating Pods yourself, causing downtime.
- No rollback — if a new version breaks, there's no built-in way to revert.

Deployments solve all of this by sitting on top of another object called a ReplicaSet, which in turn manages the Pods. You rarely interact with ReplicaSets directly — the Deployment handles that layer for you.

3. How It Works Internally

When you create a Deployment, Kubernetes creates a ReplicaSet, which then creates the actual Pods. The chain looks like this:

```
Deployment → ReplicaSet → Pods
```

When you update the Deployment (say, a new image version), Kubernetes creates a **new ReplicaSet** with the updated Pod template, gradually shifts traffic and replicas from the old ReplicaSet to the new one, and keeps the old ReplicaSet around (scaled to zero) so you can roll back instantly if needed.

4. A Basic Deployment YAML

```
apiVersion: apps/v1
kind: Deployment
metadata:
```

```

name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80

```

replicas sets how many Pod copies to keep running. **selector** tells the Deployment which Pods belong to it (matched via labels). **template** is the blueprint used to create each Pod.

5. Key Deployment Features ("the pitch")

Self-healing

If a Pod is deleted, crashes, or its node fails, the Deployment (through its ReplicaSet) automatically creates a new Pod to restore the desired replica count. You never have to babysit individual Pods.

Declarative Scaling

Change the replica count with one command and Kubernetes adds or removes Pods to match:

```
kubectl scale deployment nginx-deployment --replicas=5
```

Rolling Updates (zero-downtime deploys)

When you update the image or config, Kubernetes replaces old Pods with new ones gradually — a few at a time — instead of all at once. Users experience no downtime because some old Pods keep serving traffic while new ones start up and pass health checks.

```
kubectl set image deployment/nginx-deployment nginx=nginx:1.27
```

Instant Rollback

If a new version misbehaves, you can revert to the previous working version in seconds, because Kubernetes keeps a history of ReplicaSets:

```
kubectl rollout undo deployment/nginx-deployment
```

Pause and Resume Rollouts

You can pause a rollout mid-way to inspect a partially updated state, then resume it — useful for canary-style testing before committing to a full rollout.

Rollout History and Status Tracking

Kubernetes tracks every revision of a Deployment, so you can inspect what changed and when:

```
kubectl rollout status deployment/nginx-deployment
kubectl rollout history deployment/nginx-deployment
```

Update Strategies

- **RollingUpdate** (default) — replaces Pods gradually with no downtime.
- **Recreate** — deletes all old Pods before creating new ones (causes brief downtime, but simpler; useful when the app can't run two versions at once).

6. Common kubectl Commands

```
kubectl apply -f deployment.yaml — Create or update a Deployment from a YAML file
kubectl get deployments — List all Deployments
kubectl describe deployment <name> — See detailed status, events, and conditions
kubectl scale deployment <name> --replicas=N — Change the number of Pods
kubectl set image deployment/<name> <container>=<image> — Update the container image
kubectl rollout status deployment/<name> — Watch rollout progress
kubectl rollout undo deployment/<name> — Roll back to the previous revision
kubectl delete deployment <name> — Delete the Deployment and its Pods
```

7. When to Use a Deployment vs Other Objects

- **Deployment** — stateless apps (web servers, APIs) where Pods are interchangeable.
- **StatefulSet** — apps needing stable network identity or persistent storage per Pod (databases).
- **DaemonSet** — one Pod per node (log collectors, monitoring agents).
- **Job / CronJob** — run-to-completion or scheduled batch tasks, not long-running services.

8. Summary

A Deployment is the standard way to run and manage stateless applications in Kubernetes. It gives you self-healing, easy scaling, safe rolling updates, and instant rollbacks — all through a single declarative YAML file. Once you define the desired state, Kubernetes continuously works to keep the cluster matching it, so you spend less time on manual operations and more time shipping features.

Official reference: kubernetes.io/docs/concepts/workloads/controllers/deployment/